第 3-1 讲 • 读代码的地图与心法

> **模块 3 • 学会读代码** ｜ 建议用时 **40 分钟**（阅读 20 + 动手 20）
> 前置要求：完成模块 0、1、2（认识骨架的四大部件）

本节目标

读完并做完这一讲，你能：

1. 掌握读陌生代码的**唯一正确姿势**：**不要从头读到尾**，而是"从入口顺着追"。
2. 拿到一张**项目地图**：知道主进程那 56 个文件分五层，每层干什么。
3. 学会三个**具体的搜索技巧**：找入口、找调用者、找定义。
4. 理解"**一次只追一条线**"的原则，以及为什么同时读多个文件会让你放弃。
5. 独立完成一次**读代码实战**：找出"账户余额"是怎么算出来的。

一、先破除一个错误习惯

1.1 错误姿势：从头读到尾

大部分初学者读代码是这样的：

> 打开 `src\main\index.ts` → 从第一行开始读 → 遇到一个不认识的函数 → 跳进去看 → 里面又调用了别的东西 → 再跳进去 → **五分钟后就不知道自己从哪来的了** → 关掉，觉得自己不适合编程。

这是最糟糕的读法，原因很简单：**代码不是按"阅读顺序"组织的，是按"引用关系"组织的。**

一个真实的软件，文件之间的引用关系像一张网：

```
index.ts → window.ts → ...
    ↓          ↓
  db/index → connection.ts
    ↓
  ipc/index → ipc/records.ts → repositories/records.repo.ts → SQL
```

你顺着"跳"进去，就是在网里乱走。 走三次就迷路。

1.2 正确姿势：从入口顺着追一条线

读代码的唯一正确姿势是：

> **带着一个具体问题，从入口出发，只追一条线，追到底就停。**

关键是"一个具体问题"。比如：

- ❌ "我要读懂这个项目"——太大，必然迷路
- ✅ "我点了保存，数据是怎么写进数据库的？"——具体，能追
- ✅ "账户余额那个数字是怎么算出来的？"——具体，能追

有了问题，你就有了一把尺子：跟问题有关的代码就细看，无关的就跳过。**没有尺子，每一行都"可能有用"，你就会什么都想看，然后崩溃。**

1.3 现实比喻：查地图 vs 走迷宫

- **从头读到尾** = 蒙着眼睛在迷宫里乱走，走到哪算哪。
- **带着问题追一条线** = **先看地图找到起点和终点，然后只走那条路线。**

本讲的核心任务，就是给你这张地图。

二、项目地图

2.1 全局视图

```
记账簿\
├─ src\
│   ├─ main\       ← 后厨：56 个文件，分五层（本讲重点）
│   ├─ preload\    ← 传菜口：1 个文件
│   ├─ renderer\   ← 服务员：界面（19 个页面 + 4 个通用组件）
│   └─ shared\     ← 双方共用的类型、常量、工具
├─ scripts\        ← 开发辅助脚本（打包、测试、验证）
└─ docs\           ← 文档与截图
```

2.2 主进程的五层（这是最重要的一张图）

主进程的 **57 个** `.ts` **文件**不是平铺的，它们分成**五层**，每层有明确职责：

层	目录	职责	文件数	能不能碰数据库
① 入口层	`src\main\*.ts`	启动流程、窗口、托盘、路径	**18**	通过下层
② 接口层	`ipc\`	接住界面的请求，做参数校验	**15**	❌ 只转发
③ 仓储层	`db\repositories\`	把业务操作翻译成 SQL	**8**	✅ **主要写 SQL 的地方**
④ 服务层	`services\`	复杂业务：备份、OCR、导入导出	**9**	通过仓储层
⑤ 数据层	`db\`（连接、迁移、初始化）	数据库连接、表结构、初始数据	**15**	✅ 底层

这些数字是我实际数出来的，你可以自己验证：

```
(Get-ChildItem src\main -File -Filter *.ts | Measure-Object).Count            # 入口层 18
(Get-ChildItem src\main\ipc -File -Filter *.ts | Measure-Object).Count        # 接口层 15
(Get-ChildItem src\main\db\repositories -Filter *.ts | Measure-Object).Count  # 仓储层 8
(Get-ChildItem src\main\services -Filter *.ts | Measure-Object).Count         # 服务层 9
(Get-ChildItem src\main -Recurse -File -Filter *.ts | Measure-Object).Count   # 总共 57
```

这张表的价值：你拿到任何一个需求，都能立刻知道"该去哪一层找"。

你的问题	该去哪一层
"这个功能界面上怎么调用的"	`renderer\` + `preload\`
"这个请求怎么校验的"	`ipc\`（接口层）
"数据具体怎么存的 SQL 长什么样"	`db\repositories\`（仓储层）
"备份/OCR/导出怎么实现的"	`services\`（服务层）
"数据库表结构怎么定义的"	`db\migrations\`

2.3 为什么要有"仓储层"这一层

你可能会问：IPC 层直接写 SQL 不行吗？为什么要多一个 `repositories`？

看一个真实的对比。IPC 层的代码（`ipc\records.ts`）：

```
ipcMain.handle(IPC.recordsCreate, (_event, input: RecordInput): LedgerRecord => {
  assertReady()           // ① 确认数据库已就绪
  validate(input)         // ② 校验参数（日期格式、金额、账户是否存在……）
  return repo.create(input)   // ③ 交给仓储层去写
})
```

它自己一行 SQL 都没有。 它只做三件事：检查状态、校验输入、转发。

而仓储层（`db\repositories\records.repo.ts`）才是真正写 SQL 的地方：

```
export function create(input: RecordInput): LedgerRecord {
  const id = randomUUID()
  const ts = Date.now()
  getDb().prepare(
    `INSERT INTO records
     (id, date, amount, type, ..., created_at, updated_at)
     VALUES (?, ?, ?, ?, ..., ?, ?)`
  ).run(id, input.date, input.amount, ...)
}
```

分成两层的三个好处：

好处	说明
各管一件事	接口层管"合不合法"，仓储层管"怎么存"。改校验不会碰到 SQL
SQL 集中	全项目的 SQL 都在 `repositories\` 里，**想审计数据库操作只需看这 8 个文件**
可复用	同一个 `repo.create`，既可以被界面调用，也可以被"CSV 导入"调用（服务层）

这是一种通用的架构模式（叫"分层架构"）。你在别的项目里也会看到类似结构，可能叫 `dao`、`model`、`store`——**名字不同，思想一样：把"怎么存"和"业务逻辑"分开。**

三、三个搜索技巧

读代码 90% 的时间在**搜索**，不是在阅读。这三个技巧决定你的效率。

3.1 技巧一：找入口（从功能名反查）

场景：你想知道"预算列表"怎么实现的。

```
# 第一步：先找 IPC 名字（这是每个功能的"身份证"）
Select-String -Path src\shared\ipc.ts -Pattern "budgets"
```

输出：`budgetsList: 'budgets:list'` → 你拿到了名字 `budgets:list`。

3.2 技巧二：找调用者（谁在用这个函数）

场景：你想知道 `records.repo.ts` 里的 `create` 函数都被谁调用了。

```
Get-ChildItem src -Recurse -Include *.ts,*.tsx | Select-String "repo\.create|recordsRepo\.
```

输出：你会看到界面表单、CSV 导入服务、截图导入服务……**都在调它。**

这一步的价值：**知道"改这里会影响谁"**。这是模块 4 动手改代码前必做的检查。

3.3 技巧三：找定义（这个函数在哪写的）

场景：你看到 `assertReady()` 但不知道它干什么。

```
Get-ChildItem src -Recurse -Include *.ts | Select-String "export function assertReady"
```

输出：`src\main\ipc\guard.ts` → 打开它，**答案就在那里。**

VS Code 里的快捷方式（比命令行快得多）：

- **按住 `Ctrl` 点函数名** → 跳到它的定义
- `Alt + ←` → 跳回刚才的位置（**这个组合键一定要记住**，否则跳进去就回不来了）
- `Ctrl + Shift + F` → 全项目搜索

这三个操作就是"读代码"的全部工具。 不需要更多。

四、"一次只追一条线"原则

4.1 为什么不能同时读多个文件

初学者常犯的错：打开 5 个文件标签，跳来跳去。

问题在于"工作记忆"：人一次只能记住 5–7 个东西。你同时看 5 个文件、每个文件里有 10 个函数名，**大脑立刻过载**，然后就是那句熟悉的"我学不会"。

正确做法：

```
① 定一个问题 → ② 从入口开始 → ③ 只跟着这一个问题的线走 →
④ 遇到无关的代码，在心里说"这个先不管" → ⑤ 追到底 → ⑥ 用一句话总结你追到的路径
```

第 ⑥ 步最关键：**追完必须写下来。**哪怕只是写在纸上：

```
点击保存 → RecordForm.payload → window.api.records.create
→ preload → ipc:records:create → validate() → repo.create() → SQL INSERT
```

写下来的那一刻，这条线才真正变成你的。 这也是本讲动手任务的最后一步。

4.2 遇到看不懂的代码怎么办

三个判断：

情况	怎么办
与你的问题**无关**	**跳过。**它可能是别处的实现细节，不影响你追这条线
与你的问题**有关但是个细节**	记下来，追完主线再回来看
完全看不懂整个文件在干什么	说明你缺了前置概念 → 回去补对应模块（比如不懂 IPC 就回 2-2）

严禁的行为：在一个细节里死磕 30 分钟。**读代码是"广度优先"的活**——先把线走通，细节回头看第二遍时自然就懂了。

这也是本讲义"螺旋上升"的原因：同一个东西（比如 `repo.create`）你会在不同讲里遇到三次，每次懂更多。**第一遍不懂是正常的。**

五、动手任务

步骤 1：用地图定位（不搜索，靠推理）

问题："界面点击『立即备份』之后，文件是怎么被复制的？"

推理过程（用上面的五层表）：

1. 界面上有个按钮 → 那在 `renderer\`

2. 它要请求后厨 → 走 `preload` 的 `app.backup`
3. 接口层接住 → `ipc\app.ts`
4. 真正的文件复制逻辑 → **应该在服务层** → `services\backup.service.ts`

验证推理：

```
cd "C:\Users\34123\Desktop\记账簿"
Select-String -Path src\main\ipc\app.ts -Pattern "appBackup"
```

预期看到：`ipcMain.handle(IPC.appBackup, (): BackupResult => backupNow())`

注意：接口层这里**只有一行**——它把活交给了 `backupNow()`。然后：

```
Select-String -Path src\main\services\backup.service.ts -Pattern "export function backupNo
```

你会看到真正的实现：检查文件存在 → checkpoint（把缓存写盘）→ 复制文件。

这一步的意义：你**只靠架构推理就找到了实现位置**，没搜索。**当你能做到这一点，读代码就不再是碰运气。**

步骤 2：追一条完整链路并写下来

问题："点『保存』之后，数据怎么进数据库的？"

按顺序执行这些搜索（**每一步都要看到结果再往下**）：

```
# ① 界面侧：谁发起了请求
Select-String -Path src\renderer\src\features\records\RecordForm.tsx -Pattern "records.cre

# ② 通道侧：暴露了什么方法
Select-String -Path src\preload\index.ts -Pattern "recordsCreate"

# ③ 接口侧：接住请求并校验
Select-String -Path src\main\ipc\records.ts -Pattern "recordsCreate" -Context 0,4

# ④ 仓储侧：真正写 SQL
Select-String -Path src\main\db\repositories\records.repo.ts -Pattern "export function cre
```

然后拿出纸笔（或新建一个文本文件），写下这条线：

```
RecordForm 里点保存
  → window.api.records.create(payload)          [界面]
  → ipcRenderer.invoke('records:create', input) [preload]
  → ipcMain.handle('records:create', ...)       [主进程接口层]
      → assertReady()    确认数据库已打开
      → validate(input)  校验：日期格式、金额、账户是否存在……
      → repo.create(input)                        [仓储层]
          → INSERT INTO records ...               [SQL]
```

写下来之后，把这句话读一遍。如果你能对着它讲给别人听，**你就真正理解了记账簿最核心的一条链路。**

步骤 3：读一个校验函数（学会"读懂规则"）

```
Get-Content src\main\ipc\records.ts | Select-String "function validate" -Context 0,20
```

你会看到一串"如果不对就报错"。翻译成业务规则：

代码在检查	意思是
日期格式	日期必须是 `YYYY-MM-DD`
金额是整数且 ≥ 0	金额是"分"，不能是小数（呼应 2-3 讲）
金额不能是 0	记 0 元的账没意义
类型必须是三种之一	防止出现"第四种类型"
账户必须存在	不能记到不存在的账户上
转账时转入账户不能和转出相同	自己转给自己没意义
非转账时必须有分类	支出/收入必须归类
分类不能是已归档的	归档的分类不给用
分类类型要与记录类型一致	不能把"收入分类"用在支出上
备注不超过 200 字	防止超长文本

这个练习的价值：**校验函数是"业务规则"最集中的地方。** 你想知道"这个软件允不允许 X"，看校验代码最快。

而且你会发现：这些规则**不是凭空写的**，每一条都对应一个真实场景（自己转给自己、归档的分类、备注超长）——**这就是"读代码能读出产品设计"。**

步骤 4：验证你的地图（找出所有 SQL 的位置）

```
Get-ChildItem src -Recurse -Include *.ts | Select-String "INSERT INTO|SELECT .* FROM|UPDAT
```

我实测的结果是这样的：

文件	SQL 处数
`db\repositories\accounts.repo.ts`	24 处
`db\repositories\categories.repo.ts`	19 处
`db\repositories\records.repo.ts`	6 处
`db\repositories\budgets.repo.ts`	7 处
`db\repositories\attachments.repo.ts`	6 处
`db\repositories\recurring.repo.ts`	7 处
`db\repositories\settings.repo.ts`	6 处
`db\repositories\stats.repo.ts`	3 处
`db\migrations\002_account_archive.ts`	1 处（重建表时的取数语句）
`db\seed.ts`	2 处（首次初始化写预设数据）
`demo.ts`	1 处（写入示例数据）

结论要说得准确一点：

> **几乎全部 SQL 都在仓储层**（8 个文件，共约 78 处）。
> 例外的三个文件也**各有正当理由**：迁移文件需要重建表、`seed.ts` 要写初始数据、`demo.ts` 是造示例数据的开发工具。

这条规律的实用价值：

- **你要看"数据怎么被增删改查"→ 主要看 `db\repositories\`**（占 95%）
- **你要审"这个软件会不会动我的数据"→ 除了仓储层，还要看一眼 `seed.ts` 和迁移文件**
- **发现 SQL 出现在别的地方（比如 IPC 层或服务层）→ 那通常说明分层被破坏了**，是个值得注意的信号

这就是"读代码读出架构"：不是背规矩，而是**用一条搜索验证规矩是否被遵守**，并理解例外为什么是例外。

卡住了怎么办

现象	原因	怎么办
搜索没结果	关键词拼写和实际不同	先搜个短一点的词（如 `create`）看实际命名
追到一半断了	中间经过了一层我没讲到的文件	用"找定义"技巧跳进去，别硬猜
看不懂某个函数	正常，可能涉及没学的内容	**跳到下一步**，标记下来，第二遍再看
VS Code 里 `Ctrl+点击` 没反应	该项目可能需要索引	用 `Ctrl+Shift+F` 全项目搜索代替

六、自测题

1. 为什么不能"从头读到尾"？正确的读法是什么？

2. 主进程的五层分别是什么？"数据怎么存的 SQL"该去哪一层找？

3. 接口层（`ipc\`）为什么不直接写 SQL？分两层有什么好处？

4. 遇到一段与当前问题无关、但看不懂的代码，该怎么办？

5. 追完一条链路后，为什么必须"写下来"？

答案与解析

1. 因为**代码是按引用关系组织的，不是按阅读顺序**。从头读会不断跳进跳出于是一层层迷路。 → 正确做法：**带着一个具体问题，从入口顺着追一条线，追到底就停。**

2. ① 入口层（`src\main\*.ts`）② 接口层（`ipc\`）③ **仓储层（`db\repositories\`）** ④ 服务层（`services\`）⑤ 数据层（`db\` 连接与迁移）。 → SQL 在**仓储层**。技巧：搜 `INSERT INTO`，看结果落在哪个目录。

3. 为了让"**校验输入**"和"**怎么存数据**"分开，各管一件事。 → 好处：① 改校验不会碰到 SQL；② **全项目 SQL 集中在一层，便于审计**；③ 仓储函数能被多个调用方复用（界面、CSV 导入、截图导入）。

4. 跳过它，继续追你的线。如果它确实和你的问题有关，就记下来，追完主线再看。 → **严禁在无关细节上死磕。**读代码是广度优先的活，第一遍不懂是正常的——所以本讲义设计成螺旋上升，同一个东西你会遇到三次。

5. 因为**写下来才算真的追通了。**写的过程会暴露你其实没搞清楚的地方（比如"这里到底传了什么参数"）。 → 而且这份笔记以后能直接复用——下次遇到相关问题，你不用重新追一遍。

七、术语表

术语	一句话解释
入口 / Entry	代码开始执行的地方，追代码的起点
调用链 / Call Chain	从发起到最终执行经过的一串函数调用
分层架构	把代码按职责分成多层，每层只干一类事
接口层 / IPC 层	接住外部请求、校验参数，然后转发
仓储层 / Repository	把业务操作翻译成数据库 SQL 的那一层
服务层 / Service	实现复杂业务逻辑（备份、OCR、导入导出）
数据层	数据库连接、表结构、迁移
副作用 / Side Effect	函数除了返回值之外产生的影响（写数据库、改文件）
广度优先	先把主线走通，细节留到第二遍再看
工作记忆	人脑中"同时能记住的东西"，通常只有 5–7 项

八、本讲与项目的对应

- 你查看的文件：`ipc\app.ts`、`services\backup.service.ts`、`ipc\records.ts`、`db\repositories\records.repo.ts`
- 你产出的东西：**一张自己画的调用链笔记**（建议保留，模块 4 会用到）

- **本讲不修改任何代码，不影响你的记账数据**
- 上一讲：2-4 装修队：React 与界面 ｜ 下一讲：**3-2 追一条数据流：记一笔账**（把这条线追得更深，含错误分支）

讲义版本：v1 ｜ 对应项目版本：0.1.0